Chapitre 2:

(Prérequis)
Le langage XML
eXtensible Markup Language

hamdi.aloulou@isima.u-monastir.tn

Objectifs du cours

- Découvrir la notion et objectifs du langage XML
- Comprendre la structure et les éléments d'un document XML
- Différentier entre document bien formé et valide
- Découvrir des exemples d'application de XML

XML vs HTML

- HTML est un langage dédié pour la présentation de documents (données) sur le web.
- La demande de dynamisme et d'interactivité à afficher des limites inacceptables de HTML
  - Balisage prédéfinie dans la norme
  - Jeu de balises dédié à la mise en forme
  - Une interopérabilité limitée et un échange de contenu difficile entre application

Limites du HTML

› Exemple :

```
<H2>Bibliographie XML</H2>
<UL>
   <LI>Jean-Christophe Bernadac et François Knab, <I>Construire une application
   XML</I>, Paris, Eyrolles, 1999</LI>
   <LI>Alain Michard, <I>XML, Langage et Applications</I>, Paris, Eyrolles,
   1998</LI>
   <LI>William J. Pardi, <I>XML en Action</I>, Paris, Microsoft Press, 1999, adapté
   de l'anglais par James Guerin</LI>
</UL>
```

› Les balises html fournissent des indications pour mettre en page le texte qu'elles encadrent :

- <H2> et </H2> signalent un titre de niveau 2,
- <UL> et </UL> une liste non numérotée,
- <LI> et </LI> un élément de cette liste,

Limites du HTML

› Exemple

- Bibliographie XML
  - Jean-Christophe Bernadac et François Knab, *Construire une application XML*, Paris, Eyrolles, 1999
  - Alain Michard, *XML, Langage et Applications,* Paris, Eyrolles, 1998
  - William J. Pardi, *XML en Action*, Paris, Microsoft Press, 1999, adapté dé de l'anglais par James Guerin

› Toutes les balises d'une page HTML sont relative à sa présentation finale et à rien d'autre.

› Rien ne permet à un logiciel de comprendre le sens (la sémantique) du texte.

Limites du HTML

› Exemple

- Bibliographie XML
  - Jean-Christophe Bernadac et François Knab, *Construire une application XML*, Paris, Eyrolles, 1999
  - Alain Michard, *XML, Langage et Applications,* Paris, Eyrolles, 1998
  - William J. Pardi, *XML en Action*, Paris, Microsoft Press, 1999, adapté dé de l'anglais par James Guerin

› Dans l'exemple, il n'est pas simple de comprendre que Alain Michard est l'**auteur** d'un **livre intitulé** "XML, langage et Applications", qui est **paru** en 1998, dont l'**éditeur**, Eyrolles, est **situé** à Paris.

XML vs HTML

- L'objectif de XML n'est pas de remplacer HTML
  - **Structuration versus présentation**
    - HTML répond à la question comment présenter des données ?
    - XML répond à la question comment structurer des données ?
  - **Fond versus forme**
    - HTML s'intéresse à la forme
    - XML s'intéresse au fond
- XML et HTML sont complémentaires :
  - L'évolution du web va vers une séparation du fond et de la forme
  - Il faudra conserver un langage de mise en forme style HTML

XML (e**X**tensible **M**arkup **L**anguage)

- XML est un langage balisé qui peut être défini comme étant un langage standardisé de **description** et d'**échange** de **données structurés**.
- XML est un langage universel pour les données sur le Web qui permet au développeur de **délivrer du contenu depuis les applications à d'autres applications.**
- Un standard d'échange
  - Lisible : texte balisé avec marquage
  - Clair : séparation du fond et de la forme
  - Extensible : supporte les évolutions applicatives

Les objectifs du format XML

- Les documents doivent être faciles à créer
- La définition libre de nouvelles balises et de nouveaux attributs
- Les balises ont un sens et une hiérarchie
- La vérification de la structure d'un document (en utilisant les schémas XML)

Exemple

› HTML

```
<H2>Bibliographie XML</H2>
<UL>
   <LI>Jean-Christophe Bernadac et François Knab, <I>Construire une
   application XML</I>, Paris, Eyrolles, 1999</LI>
   <LI>Alain Michard, <I>XML, Langage et Applications</I>, Paris, Eyrolles,
   1998</LI>
   <LI>William J. Pardi, <I>XML en Action</I>, Paris, Microsoft Press,
   1999, adapté de l'anglais par James Guerin</LI>
</UL>
```

Exemple

› Voici ce que donnerait la même page si elle était codée en XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<BIBLIO SUBJECT="XML">
     <BOOK ISBN="9782212090819" LANG="fr" SUBJECT="applications">
          <AUTHOR>
               <FIRSTNAME>Jean-Christophe</FIRSTNAME>
               <LASTNAME>Bernadac</LASTNAME>
          </AUTHOR>
          <AUTHOR>
               <FIRSTNAME>François</FIRSTNAME>
               <LASTNAME>Knab</LASTNAME>
          </AUTHOR>
          <TITLE>Construire une application XML</TITLE>
          <PUBLISHER>
               <NAME>Eyrolles</NAME>
               <PLACE>Paris</PLACE>
          </PUBLISHER>
          <DATEPUB>1999</DATEPUB>
     </BOOK>
```

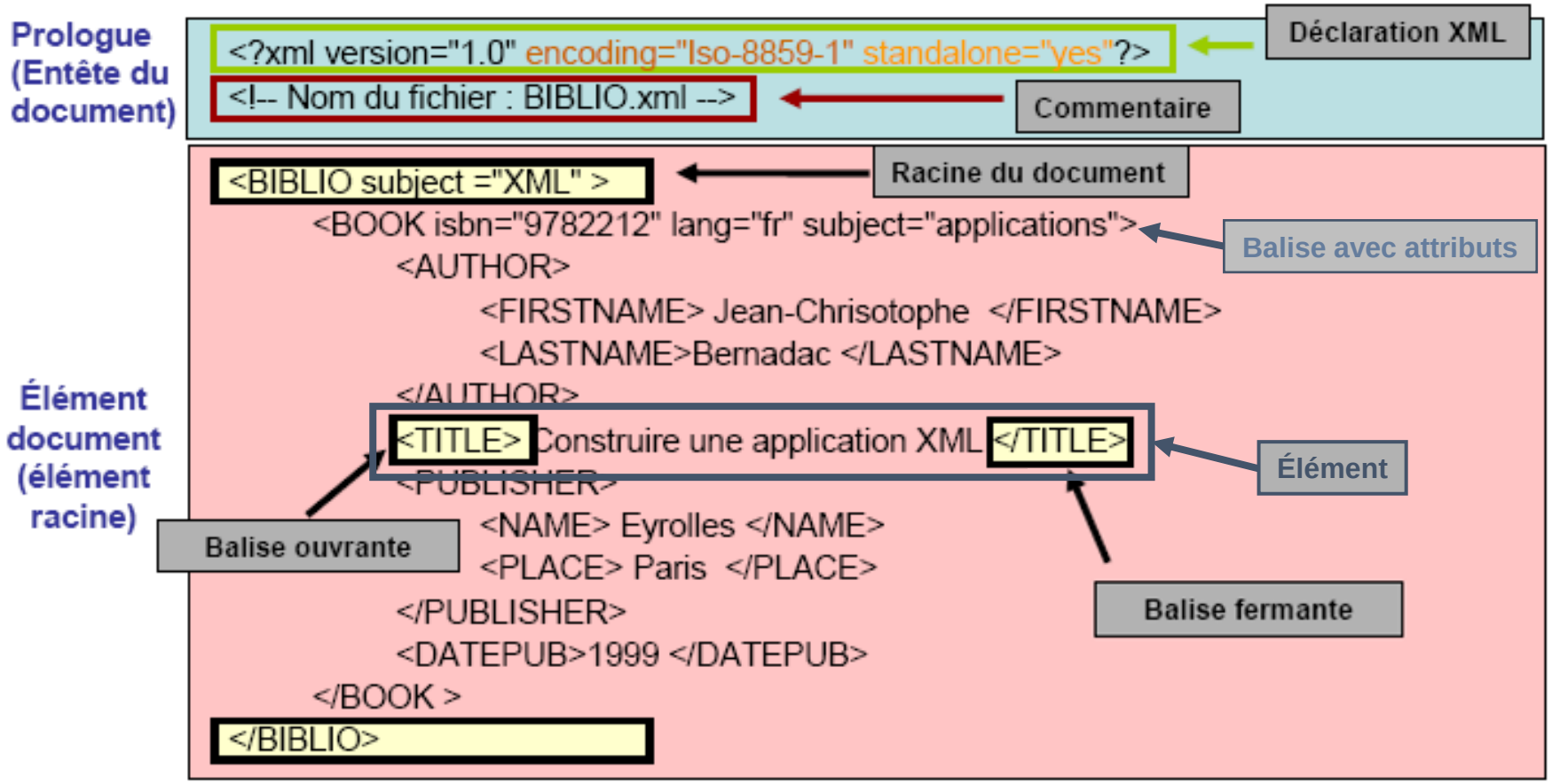
Exemple

```
<BIBLIO>
    <BOOK ISBN="9782212090529" LANG="fr" SUBJECT="général">
        <AUTHOR>
            <FIRSTNAME>Alain</FIRSTNAME>
            <LASTNAME>Michard</LASTNAME>
        </AUTHOR>
        <TITLE>XML, Langage et Applications</TITLE>
        <PUBLISHER>
            <NAME>Eyrolles</NAME>
            <PLACE>Paris</PLACE>
        </PUBLISHER>
        <DATEPUB>1998</DATEPUB>
    </BOOK>
    <BOOK ISBN="9782840825685" LANG="fr" SUBJECT="applications">
        <AUTHOR>
            <FIRSTNAME>William J.</FIRSTNAME>
            <LASTNAME>Pardi</LASTNAME>
        </AUTHOR>
        <TRANSLATOR PREFIX="adapté de l'anglais par">
            <FIRSTNAME>James</FIRSTNAME>
            <LASTNAME>Guerin</LASTNAME>
        </TRANSLATOR>
        <TITLE>XML en Action</TITLE>
        <PUBLISHER>
            <NAME>Microsoft Press</NAME>
            <PLACE>Paris</PLACE>
        </PUBLISHER>
        <DATEPUB>1999</DATEPUB>
    </BOOK>
</BIBLIO>
```

Exemple

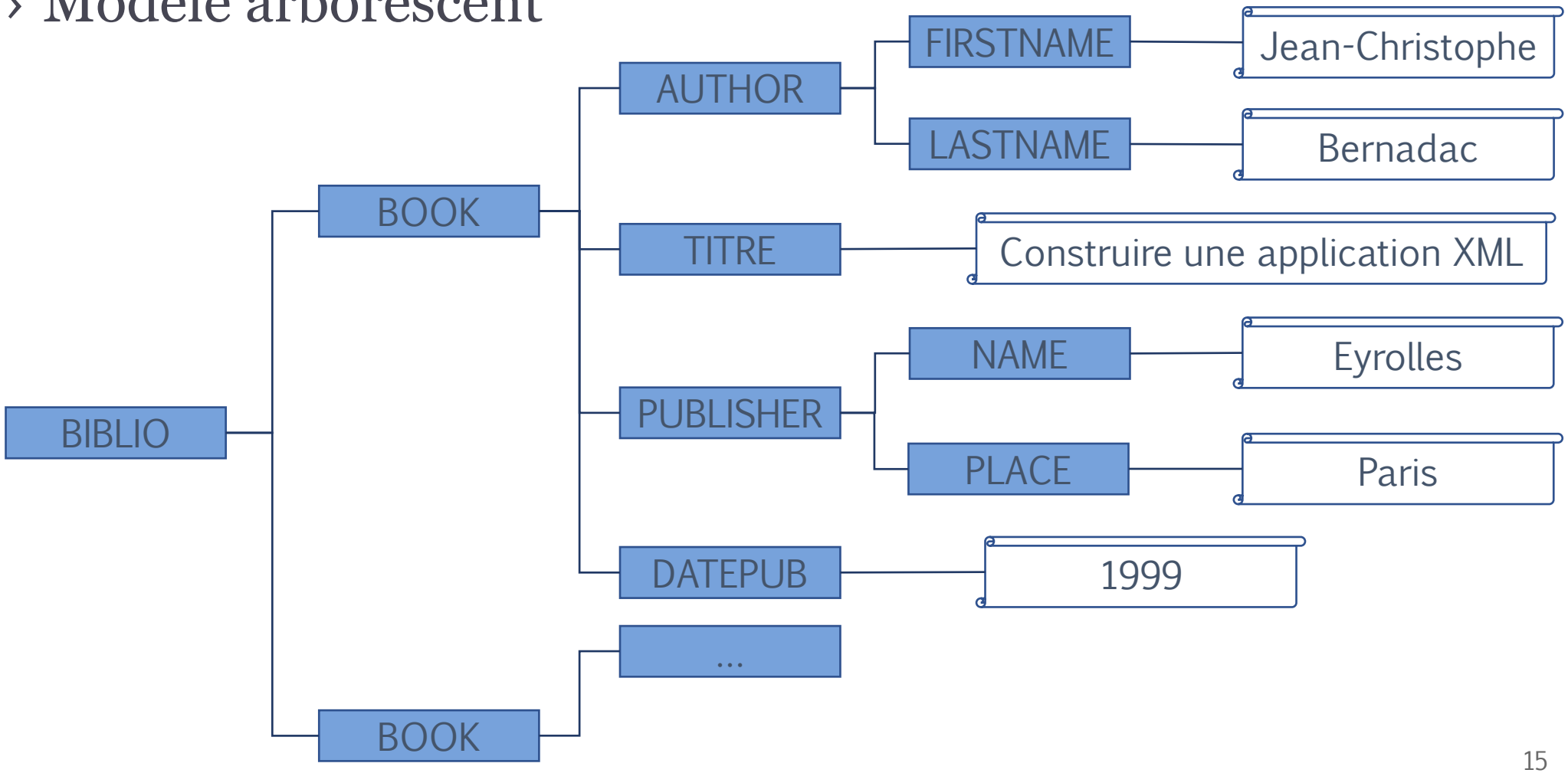
- L'élément auteur comprend le prénom (balise "firstname") et le nom (balise "lastname") de l'auteur.
- Des informations supplémentaires (langue, sujet, ISBN), ont pu être ajoutées sous la forme d'attributs situés à l'intérieur même des balises.

Description d'un document XML



Description d'un document XML

› Modèle arborescent



Description d'un document XML

- Le **prologue** d'un document XML comprend :
- La déclaration XML et la version

```
<?xml version="1.0"?>
```

Elle est optionnelle mais recommandée, notamment pour y faire figurer le numéro de version.

- L'encodage par l'attribut encoding

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

Il permet d'indiquer la représentation physique des caractères du fichier; afin d'éviter de coder chaque caractère spécial.

- L'attribut SDD Standalone Document Declaration

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="yes"?>
```

Il permet de déterminer si le document est autonome (ou autodescriptif). Si ce n'est pas le cas, c'est qu'il fait référence à une DTD ou un schéma XML externe.

Description d'un document XML

- Les attributs **version, encoding et standalone qui figurent** dans la déclaration doivent y figurer dans cet ordre.
- Les commentaires

```
<!-- Nom du fichier : BIBLIO.xml -->
```

  - Ils peuvent également exister,
  - Ils ne peuvent être placés à l'intérieur d'une balise et ne sont pas obligatoirement transmis à l'application.

Description d’un document XML

- **Les éléments syntaxiques XML** forment la seconde partie d'un document XML, correspond à un élément unique, élément racine (comparable au <BODY> en HTML) qui peut contenir d'autres éléments de données.
- Il existe trois éléments syntaxiques XML :
  - Les éléments
  - Les attributs
  - Les entités

Les éléments

- Un élément s'ouvre et se ferme par une balise
- Le nom de l'élément est repris dans la balise ouvrante et dans la balise fermante.



Les attributs

- L'attribut se trouve dans la balise ouvrante d'un élément
- L'attribut n'est pas repris dans la balise fermante
- Un élément peut contenir plusieurs attributs
- Un même attribut ne peut être présent qu'une seule fois dans un élément
- L'ordre des attributs n'a pas d'importance au sein d'un élément
- La valeur de l'attribut est indiquée entre guillemets

```
<quantite unite ="g" >100</quantite>
```

Les entités

- Certains caractères ont un sens particulier en XML (caractères spéciaux)
  - Exemple: >, &, "
- Les entités ont été prédéfinies afin de pouvoir utiliser les caractères réservés « **&entite;** »
- Une entité est une chaîne de caractère commençant par & et se terminant par ;
- Une entité est remplacée par la chaîne de caractère qu'elle représente.

Les entités

- **Les entités prédéfinies**

Caractère	Entité
&	&
<	<
>	>
"	"
'	&apost;

- Exemple:

```
<message>salaire &lt; 1000</message>
```

Exercice

› Soit le document suivant représentant un email :

Date: Mar, 28 Oct 2003 14:01:01 +0100 (CET)

De : Marcel marcel@ici.fr

A : Robert robert@labas.fr

Sujet: Hirondelle

Salut, Pourrais-tu m'indiquer quelle est la vitesse de vol d'une hirondelle transportant une noix de coco ?

A très bientôt,

Marcel

› Proposer une représentation XML de cette email.

Solution

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<email>
   <entete>
        <date type="JJMMAAAA">28102003</date>
        <heure type="24" local="(GMT+01 :00)">14:01:01</heure>
        <expediteur>
                <adresse mail="marcel@ici.fr">Marcel</adresse>
        </expediteur>
        <recepteur>
                <adresse mail="robert@labas.fr">Robert</adresse>
        </recepteur>
        <sujet>Hirondelle</sujet>
   </entete>
   <corps>
        <salutation>Salut,</salutation>
        <paragraphe>Pourrais-tu m'indiquer quelle est la vitesse de vol d'une
        hirondelle transportant une noix de coco ?</paragraphe>
        <politesse>A très bientôt,</politesse>
        <signature>Marcel</signature>
    </corps>
</email>
```

Structure du XML: Document bien formé (1/3)

- La syntaxe XML comprend des règles strictes. Dans un document XML **bien formé :**
  - Il doit y avoir une balise racine (qui englobe toutes les autres, BIBLIO dans l'exemple).
  - Toute balise ouvrante doit être associée à une balise fermante, sauf l'élément sans contenu

    (exemple: **<BOOK/> = <BOOK> </BOOK>**).

Structure du XML: Document bien formé (2/3)

- La syntaxe XML comprend des règles strictes. Dans un document XML **bien formé :**
  - Il ne doit pas y avoir d'entrelacement de balises :

    <a> (syntaxe incorrecte).
  - La casse (majuscules/minuscules) doit être respectée.
  - Les noms des attributs doivent être notés en minuscule.
  - Les valeurs des attributs doivent être notés entre guillemets ou apostrophes.

Structure du XML: Document bien formé (3/3)

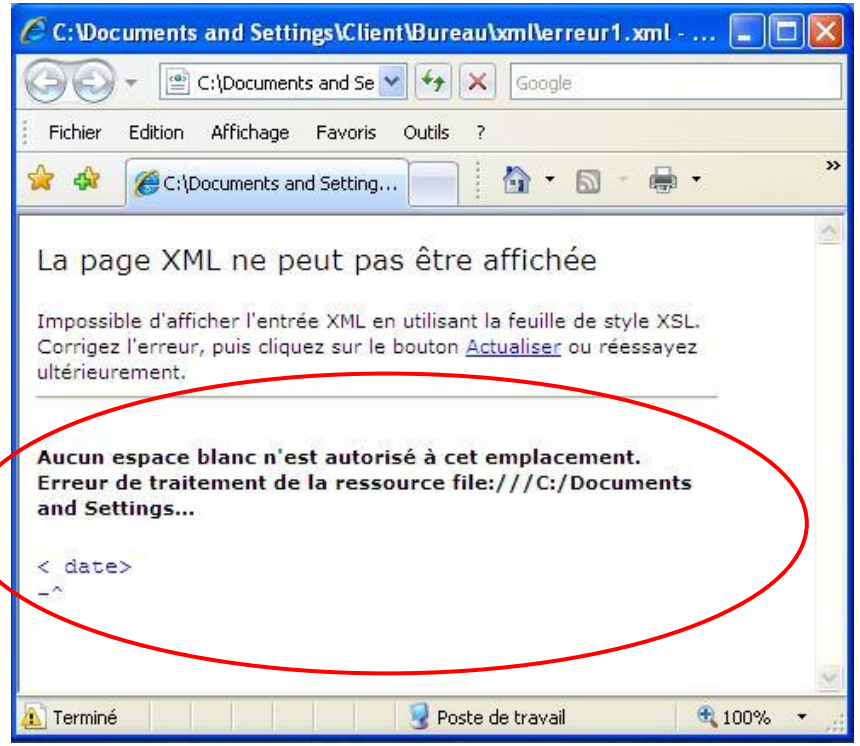
- La syntaxe XML comprend des règles strictes. Dans un document XML bien formé :
  - Les caractères "<" et "&" sont interdits dans les contenus et remplacés par les entités "<" et "&".
  - L'usage de ">" ou des guillemets simples ou doubles peut également être perturbant. Dans ce cas, on a recours à "> ", "' " et "" ".
  - Aucun espace après le caractère d'ouverture "<" ou avant caractère de fermeture ">"

Document bien formé

- Le non-respect des conditions précédentes entraînera systématiquement une erreur.

- Soit le code erroné suivant :

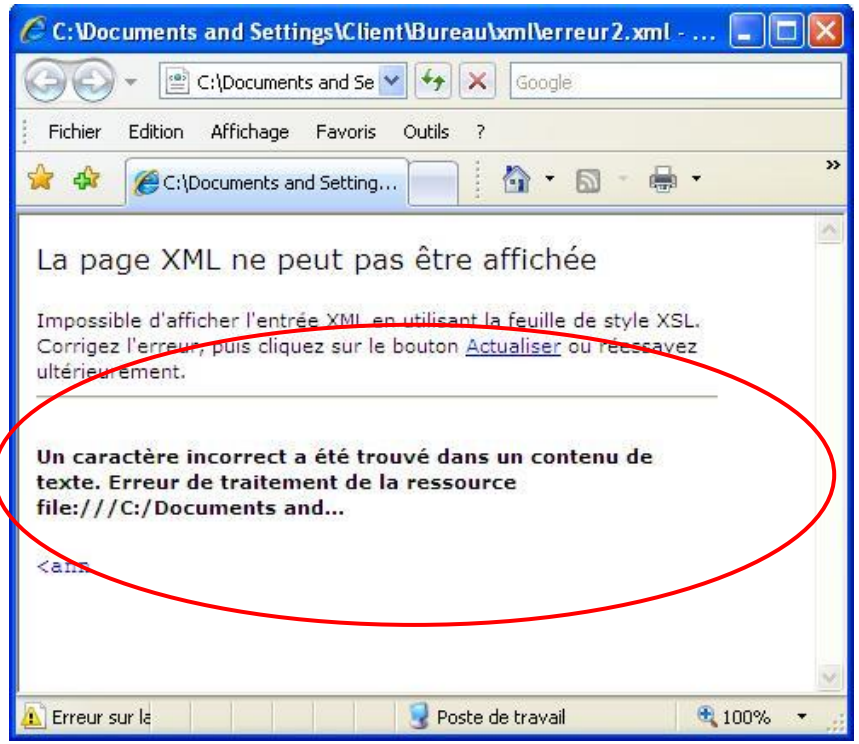
```
<?xml version="1.0" standalone="yes"?>
< date>
<mois> 09 </mois>
<année> 2007 </année>
</data>
```



Document bien formé

Soit le code erroné suivant :

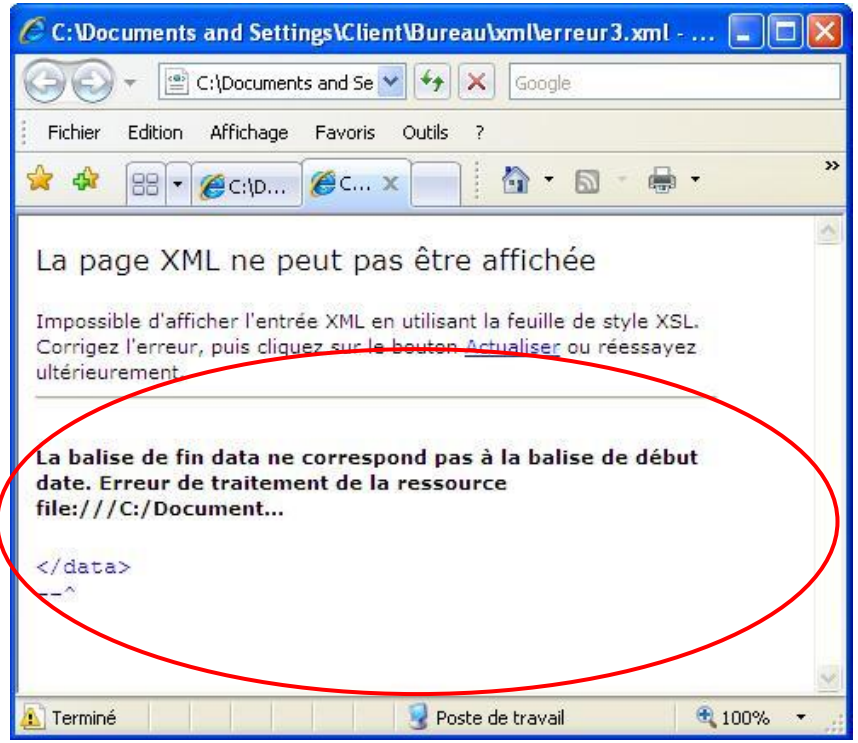
```
<?xml version="1.0" standalone="yes"?>
<date>
<mois> 09 </mois>
<année> 2007 </année>
</data>
```



Document bien formé

Soit le code erroné suivant :

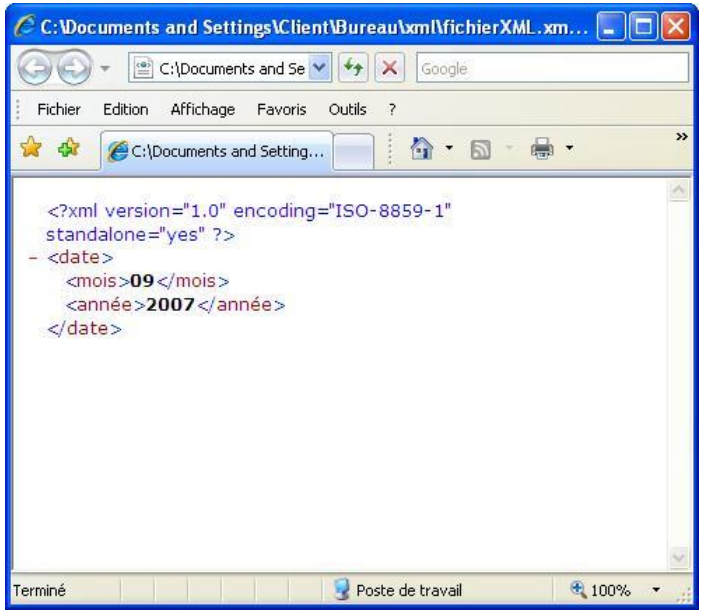
```
<?xml version="1.0" encoding="ISO-8859-1"
  standalone="yes"?>
<date>
<mois> 09 </mois>
<année> 2007 </année>
</data>
```



Document bien formé

- Le non-respect des conditions précédentes entraînera systématiquement une erreur.
- Après une troisième correction (rectification du nom de la balise de fermeture), on aboutit à un document bien formé.

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="yes"?>
<date>
<mois> 09 </mois>
<année> 2007 </année>
</date>
```



Exercice

› Observez le document XML suivant :

```
<?xml version="1.0"?>
<!-- this is a note -->
<note date=3 janvier>
    <to>Bob</To>
    <from>Alice</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
</note>
<note date="5 janvier" <!-- this is another note --> >
    <to>Alice</to>
    <from>Bob
    <body>No problem & see you soon</body>
</note>
<note/>
```

› 1. Ce document est-il bien formé (i.e. respecte-t-il la syntaxe XML) ?

› 2. S'il ne l'est pas, corrigez les erreurs.

Solution

```
<?xml version="1.0"?>
<!-- this is a note -->
<document>
    <note date="3 janvier">
        <to>Bob</to>
        <from>Alice</from>
        <heading>Reminder</heading>
        <body>Don't forget me this weekend!</body>
    </note>
    <note date="5 janvier">
        <!-- this is another note -->
        <to>Alice</to>
        <from>Bob</from>
        <body>No problem &amp;amp; see you soon</body>
    </note>
    <note/>
</document>
```

Validité des documents

- Document bien formé (Well Formed document)
  - Balises correctement imbriquées
  - Analysable (parsable) et manipulable
  - Pas nécessairement valide par rapport à un schéma
- Document valide (Valid document)
  - Bien formé
  - Conforme à un schéma

Contrôle du XML: Document valide

- XML n'est vraiment puissant que si la cohérence est contrôlée
- Pour cela, deux technologies au choix:
  - **DTD**
    - Suffisant dans la plupart des cas / Syntaxe différente de XML
  - **XML-Schéma**
    - Puissant (données plus fortement typées) / Syntaxe de style XML
- DTD et schéma permettent de définir son propre langage basé sur XML
  - Vocabulaire (balises)
  - Grammaire (imbrications)

Exemples d'applications XML

- MathML : les maths en XML
  - Permettre l'échange et le traitement d'expressions mathématiques sur le Web
  - Insertion aisée d'expressions mathématiques dans des documents HTML ou XML
  - Communication d'expressions entre applications au plan s*émantique*
- SVG : Scalable Vector Graphic
  - Langage de description de graphiques 2D
  - Graphiques vectoriels
  - Interactifs et dynamiques



Exemples d'applications XML

- SOAP : Simple Object Access Protocol
  - Protocole d'échange de données entre applications distantes
  - Adapté pour être utilisé au-dessus du protocole HTTP (méthode POST)
  - Structure d'un message SOAP
    - Enveloppe Envelope; Entête Header; Corps Body
- REST : **RE**presentational **S**tate **T**ransfert
  - Protocole d'échange de données entre applications distantes
  - Fortement basé sur HTTP (méthodes GET, POST, PUT, DELETE)
  - Peut utiliser XML comme format d'échange de données entre applications.